07 · Herencia y polimorfismo

1° ¿Qué imprime este programa de herencia?
R = d. Clase Padre Clase Hija
El constructor de la superclase se ejecuta antes que el cuerpo del constructor de la subclase.

2° Given the following classes, which line of code correctly creates an instance of the Dog class?
R = a.Dog dog = new Dog();
b.Animal dog = new Dog();
Una referencia de la clase padre puede apuntar a un objeto de una clase hija: Animal a = new Dog();. El tipo del objeto lo determina new.

3° Which statement, when inserted into line " // TODO code application logic here", is valid in compilation time change?
R = b.sc = asc;
Puedes asignar hija → padre sin cast: padre = hija;. Se copia la referencia; no se transforma ni se crea otro objeto.

4° ¿Cuál será el resultado de ejecutar el siguiente código?
R = c.Child
En métodos de instancia sobrescritos, el tipo de referencia permite la llamada y el tipo real del objeto determina qué implementación se ejecuta.

5° ¿Cuál será el resultado del siguiente fragmento de código?
R = d.Bark
Un método de instancia sobrescrito se ejecuta según el objeto real, no según el tipo de la variable. Animal a = new Dog(); a.makeSound(); → Bark.

6° ¿Cuál será la salida del siguiente código?
R = b.Ladrido.
Para un método de instancia sobrescrito, mira el objeto creado con new: Animal miAnimal = new Perro(); → se ejecuta el método de Perro.

7° ¿Qué muestra en pantalla al ejecutar este código?
R = a.Child
El cast cambia cómo se trata una referencia, no el objeto. Un cast hacia una subclase solo funciona si el objeto real es compatible con ella.

8° ¿Cuál es el resultado del siguiente código?
R = b.sc: class SampleClass asc: class AnotherSampleClass
getClass() devuelve la clase real del objeto, independientemente del tipo declarado de la variable.

9° What is the result?
R = d. sc: class AnotherSampleClass asc: class AnotherSampleClass
sc = asc copia la referencia. Ambas variables apuntan al mismo objeto y sus llamadas a getClass() devuelven la misma clase real.

10° ¿Cuál es el resultado del siguiente código?
R = c.Hash is: 111111, 44444444, 999999999
Al crear los objetos de las clases hijas, primero se ejecuta el constructor del padre y después el de la hija. Estos constructores no imprimen nada.
Aunque varias referencias tengan el mismo tipo padre, cada objeto ejecuta su propia versión del método de instancia sobrescrito.

11° What will be the output of the following code snippet?
R = a.A's constructor B's constructor C's constructor
En una cadena C extends B y B extends A, al hacer new C() las impresiones de estos constructores ocurren en orden A → B → C. Si no escribes this(...) ni super(...), Java inserta super().

12° Que imprime en pantalla este código?
R = b.It will print "Base constructor with message: Hello" followed by "Derived constructor".
super(argumentos) llama al constructor compatible del padre; no llama también al constructor sin argumentos. Primero termina esa llamada y después continúa el constructor hijo.
super("Hello") selecciona el constructor del padre que recibe un String. No se ejecutan los dos constructores de Base.

13° What is the result?
R = d.Compilation fails.
La asignación directa hija → padre es válida; padre → hija no. Primero verifica que compile y después analiza qué método se ejecutaría.
Beta extends Alpha y Gamma extends Beta. Una variable de tipo Gamma no puede recibir directamente objetos de sus clases padre.

14° What will happen when you try to compile the following code?
R = c.Compilation error.
Al sobrescribir, puedes mantener o ampliar el acceso, nunca reducirlo. Un método protected solo puede sobrescribirse como protected o public.

15° What is the result?
R = a.message middle end
Si una clase no declara constructores, Java genera uno por defecto que llama a super(). Para que compile, el padre debe tener un constructor sin argumentos accesible.

16° Which three methods, inserted individually at line, will correctly complete class Two(Choose three)?
R = b.void foo() { /* more code here */ }
c.public void foo() { /* more code here */ }
e.protected void foo() { /* more code here */ }
Tiene retorno void y acceso de paquete porque no lleva modificador.
Para sobrescribirlo, la hija debe conservar el retorno void y mantener o ampliar su acceso.
Al sobrescribir un método void con acceso de paquete, conserva void y usa acceso de paquete, protected o public; nunca private.

17° ¿Qué métodos de anulación (Override) son válidos para las siguientes clases?
R = c.void display() { System.out.println("InChild"); }
d.public void display() { System.out.println("Child"); }
Para sobrescribir un método de instancia con acceso de paquete, conserva un retorno compatible, no usar static y mantener o ampliar el acceso.

18° Which line of code, when inserted at the commented line, causes the compiler to fail?
R = d.public char met() { return '2'; }
Cambiar los parámetros permite sobrecargar y usar otro retorno. Cambiar solo el retorno no crea una sobrecarga y falla si es incompatible con el método heredado.

19° ¿Cuál de las siguientes afirmaciones describe con precisión las relaciones que pueden existir entre clases en Java?
R = a.La herencia representa una relación "es-un" donde una clase deriva de otra clase.
b.La composición representa una relación "tiene-un" donde una clase contiene una instancia de otra clase.
d.Tanto la herencia como la composición pueden usarse juntas para modelar relaciones complejas.
e.El uso (o asociación) representa una relación "usa-un" donde una clase utiliza métodos o instancias de otra clase.
f.Se debe preferir la composición sobre la herencia para fomentar la reutilización de código y la flexibilidad.
Herencia = **es-un**; composición = **tiene-un**; uso = **usa-un**. Prefiere composición para reutilizar comportamiento con flexibilidad y herencia cuando exista una relación válida **es-un**.

20° What is the result?
R = d.The Dog barks
Una clase abstracta no se instancia directamente, pero puede tener constructores, métodos concretos y métodos estáticos. Animal a = new Dog() es válido y ejecuta los métodos sobrescritos de Dog.

21° What is the result?
R = a.Compilation fails.
Un método public solo puede sobrescribirse como public. Antes de aplicar polimorfismo, verifica que la sobrescritura compile.

22° Dado el siguiente código. ¿Cuál será el resultado?
R = c.Java Jeve va
Los métodos sobrescritos no se encadenan automáticamente: se ejecuta la versión del objeto real. Para llamar a la versión del padre se usa super.metodo().

23° Consider the following Java Code: What will be the output of the above code?
R = b.Sound Roar Bark Growl
La sobrecarga se selecciona al compilar según el tipo de referencia y los argumentos; la sobrescritura se resuelve al ejecutar según el objeto real.

24° What is the result?
R = c.Feline cougar c c
Si la hija no declara otro campo del mismo nombre, this.campo y super.campo acceden al mismo campo heredado. super no recupera el valor original.

25° ¿Cuál de las siguientes anotaciones es verdadera acerca de la siguiente jerarquía de clases?
R = a.ElectricCar myCar = new ElectricCar(); System.out.println(myCar.type); // Salida: ElectricCar
Los **campos se ocultan** y se seleccionan por el tipo de referencia; los **métodos de instancia sobrescritos** se ejecutan según el objeto real.

26° What is the result?
R = d.Compilation fails because of an error in line 18.
Si el padre solo tiene constructores con argumentos, la hija debe llamar a uno mediante super(argumentos) o encadenar con otro constructor suyo que lo haga. Java no crea un constructor vacío cuando ya declaraste alguno.

27° What is the result?
R = f.Atom granite atom granite.
Cada new crea un objeto y ejecuta su cadena de constructores. super(...) inicializa la parte heredada del objeto actual; new Padre(...) crea otro objeto independiente.

28° What is the result?
R = a.1234
this(...) y super(...) ejecutan el constructor llamado antes de continuar con las siguientes instrucciones del constructor actual.

29° What should statement1, statement2, and statement3, be respectively, in order to produce the result?
Shape: constructor
Shape: foo
Square: foo
R = d.Square square = new Square (); square.foo (); square.foo("bar");
Los argumentos seleccionan la sobrecarga. Dentro de un método sobrescrito, super.metodo() llama explícitamente a la implementación del padre.

30° ¿Cuál de los siguientes fragmentos de código generará una ClassCastException en tiempo de ejecución?
R = d.d
El objeto real es un A, y el cast no puede convertirlo en un B**. Recordar: todo B es un A, pero no todo A es un B.**

1. new A() crea un objeto de tipo A.
2. ob guarda su referencia.
3. (B) ob compila porque existe una relación de herencia entre A y B.
4. Al ejecutar, Java comprueba si el objeto es compatible con B.
5. Como el objeto es únicamente un A, lanza ClassCastException. La asignación a b no se completa.

31° Which three are valid? (Choose three).
R = b.p2 = (ClassC)p4;
d.p1 = (ClassB)p3;
e.p0 = p1;
Hija → padre: asignación directa. Padre → hija: requiere cast y un objeto compatible. Entre clases hermanas, el cast directo no compila.

32° Which is true?
R = c.If line 12 is removed, the compilation succeeds.
Primero revisa si el cast es válido entre los tipos declarados; después, si el objeto real es compatible. Un cast puede compilar y aun así lanzar ClassCastException.

33° Dado el siguiente código: ¿Cuál es el resultado de la ejecución del programa?
R = d.The code runs with no output.

Instrucción	Qué sucede	Resultado
Gamma g = new Epsilon();	Crea un Epsilon y guarda su referencia como Gamma.	Hija → padre.
Omega o = (Omega) g;	Comprueba si el objeto implementa Omega. Como es un Epsilon, sí la implementa.	Cast válido.
Epsilon e = (Epsilon) g;	Comprueba si el objeto es compatible con Epsilon. Es exactamente un Epsilon.	Cast válido.
Omega m = e;	Asigna la referencia a una variable de la interfaz que Epsilon implementa.	No requiere cast.

Un objeto es compatible con su clase, sus superclases y las interfaces que implementa. El cast comprueba esa compatibilidad; no transforma el objeto.

34° Cuál es el resultado?
R = b.true
`instanceof` devuelve `true` si el objeto es compatible con el tipo consultado. Con una referencia `null`, siempre devuelve `false`.

35° What is the result?
R = g.Compilation fails
El padre no hereda miembros de la hija. El tipo de referencia determina qué miembros puedes acceder; el polimorfismo no hace aparecer campos que el padre no declara.

36° Which two, independently, will allow Sub to compile? (Choose two)
R = b.Change line 13 to: public Sub() { super(5); }
d.Change line 13 to: public Sub() { this(5); }
Revisa por separado el acceso a los campos y la llamada al constructor padre. `super(5)` llama al padre; `this(5)` llama a otro constructor de la misma clase, que debe terminar encadenando con el padre.

37° Which five methods, inserted independently at line 5, will compile? (Choose five).
R = a.protected long blipvert(long x) { return 0;
b.public int blipvert(int x) { return 0; }
c.private int blipvert(long x) { return 0; }
d.protected long blipvert(int x, int y) { return 0; }
f.protected int blipvert(long x) { return 0; }

Opción	Tipo	¿Compila?	Motivo
a. `protected long blipvert(long x)`	Sobrecarga	✅	Cambia el parámetro a `long`. Puede tener otro retorno.
b. `public int blipvert(int x)`	Sobrescritura	✅	Conserva el retorno `int` y amplía el acceso a `public`.
c. `private int blipvert(long x)`	Sobrecarga	✅	Cambia el parámetro a `long`. Al ser otro método, puede ser `private`.
d. `protected long blipvert(int x, int y)`	Sobrecarga	✅	Recibe dos parámetros, por lo que puede devolver `long`.
e. `protected long blipvert(int x)`	Sobrescritura	❌	No puede cambiar el retorno primitivo de `int` a `long`.

f. `protected int blipvert(long x)`	Sobrecarga	✅	Cambia el parámetro a `long`.
g. `private int blipvert(int x)`	Sobrescritura	❌	Reduce el acceso de `protected` a `private`.

Si cambian los parámetros, es sobrecarga y puede cambiar el acceso y el retorno. Si los parámetros son iguales, es sobrescritura: no reduzcas el acceso y conserva exactamente el retorno si es primitivo.

38° Which three lines will compile and output "Right on!"?
R = c.Line 19
d.Line 20
f.Line 22
Para convertir una referencia antes de llamar a un método, usa `((Tipo) referencia).metodo()`. El tipo de referencia debe permitir la llamada y el objeto real debe ser compatible con el cast.

39° Dado el siguiente código. ¿Cuál es el resultado?
R = c.Valor de la clase C 28
Método sobrescrito: se ejecuta según el objeto real → `C`.
Campo: se selecciona según el tipo de referencia → `A`.
Métodos sobrescritos → objeto real. Campos → tipo de referencia. Modificar el campo de la hija no modifica los campos del mismo nombre declarados en sus padres.

40° What is the result?
R = c.The Dog barks
Los métodos `private` no se heredan ni se sobrescriben. Un método con la misma firma en la hija es independiente; no aplica polimorfismo sobre el método privado del padre.

08 · Clases abstractas, interfaces, enums y anidadas

1° ¿Cuál de las siguientes declaraciones de clase no es correcta?
d.abstract final class Person { }
Una clase puede ser abstract o final, pero nunca ambas. Una clase abstracta no necesita tener métodos abstractos.

2° Which two statements are true?
R = a.A class that is abstract cannot be instantiated or turned into an object (instance).
c.If a class has abstract methods, then it must be specified as an abstract class.
Una clase extiende una clase e implementa interfaces; una interfaz solo extiende interfaces. Si una clase deja métodos abstractos sin implementar, debe ser abstract.

3° Which two statements are true?
R = c.An abstract class can be extended by a concrete class.
d.An abstract class can implement an interface
Una clase abstracta puede extender otra clase e implementar interfaces, dejando métodos abstractos pendientes. La primera subclase concreta debe tener implementados todos esos métodos.

4° What is true about the class Wow?
R = a.It compiles without error.
Una clase abstracta puede tener atributos, constructores con parámetros y métodos privados concretos; no está obligada a tener métodos abstractos ni un constructor vacío.

5° Which statement is true about the following code?
R = a.The program will print "Drawing a circle" followed by "This is a shape".
Un método abstracto termina en ;. La subclase lo implementa al sobrescribirlo con un cuerpo { ... }; los métodos concretos de la superclase pueden heredarse sin reescribirlos.

6° What will be the output of the following code?
R = a.Car started Vehicle stopped Bike started Vehicle stopped
El tipo abstracto puede usarse como referencia. Un método sobrescrito se ejecuta según el objeto real; un método concreto no sobrescrito conserva su implementación heredada.

7° What will be the output of the following code snippet?
R = c.MONDAY TUESDAY WEDNESDAY THURSDAY FRIDAY SATURDAY SUNDAY
MiEnum.values() devuelve todas las constantes en orden de declaración. Los separadores de la salida dependen de lo que escribas en print().

8° Which of the following code snippets will result in a compilation error when implementing the Vehicle interface?
R = Opción b:

```
public class Bike implements Vehicle {
    public void start() {
        System.out.println("Bike starts");
    }
    void stop() {
        System.out.println("Bike stops");
    }
}
```

Los métodos abstractos de una interfaz son public: su implementación no puede reducir el acceso. Una clase que implementa una interfaz sí puede añadir métodos propios.

9°Which code fragment is illegal?
R = d.class Base3 { abstract int var1 = 89; }
abstract puede aplicarse a clases y métodos, **nunca a campos ni constructores**. Una clase concreta puede contener una clase interna abstracta.

10° Given: Which method can be declared within the C class to make it compile?
R = e.None of the options is valid.
Para implementar un método abstracto de una interfaz en una clase concreta, escribe public y un cuerpo { ... }. Sin modificador, el método de la clase tiene acceso de paquete.

11° What is the result?
R = d.Compilation fails
Las implementaciones de métodos abstractos de interfaces deben ser públicas, y Java distingue weight de Weight.

12° Dado el siguiente código. ¿Cuál es el resultado?
R = b.Amarillo Rosa Rojo
El tipo de referencia determina qué métodos puedes llamar; el objeto real determina qué implementación sobrescrita se ejecuta. Un método abstracto puede llamarse mediante la referencia si el objeto concreto lo implementa.

13° What is the result?
R = d.red green
Entre constantes del mismo enum puedes usar == o equals(). Para comparar con un String, usa constante.name().equals("NOMBRE"); constante.equals("NOMBRE") devuelve false.

14° ¿Cuáles son los modificadores que puede tener una clase anidada en Java?
R = h.Todas las opciones son correctas
Una clase anidada miembro admite todos los modificadores de la lista, pero solo en combinaciones compatibles. static elimina la necesidad de una instancia exterior; no impide crear objetos de la clase anidada.

09 · Excepciones

1° In Java, which class is the superclass of all errors and exceptions?
R = e.Throwable
La clase base de todos los errores y excepciones es **Throwable**; sus dos ramas principales son **Error y Exception**.

2° ¿Cuáles son los tipos de excepciones en Java?
R = c.Checked
d.Unchecked
Checked → debes capturar o declarar. Unchecked → puedes hacerlo, pero el compilador no te obliga.

3° ¿Cuáles de las siguientes son palabras reservadas en Java para el manejo de excepciones?
R = a.try, catch, finally, throw, throws

Palabra	Función
try	Delimita el bloque cuya ejecución puede lanzar una excepción.
catch	Captura una excepción compatible con el tipo declarado.
finally	Define un bloque que se ejecuta al salir del try/catch, normalmente para limpieza.
throw	**Lanza** una excepción concreta: throw new IOException();.
throws	**Declara** en la firma de un método las excepciones que puede propagar: void leer() throws IOException.

4° ¿Para qué sirve la palabra clave throw en Java?
R = d.Crear y lanzar explícitamente una excepción.
class ... extends Exception define; new crea; throw lanza; throws declara; catch captura.

5° ¿Cuál de las siguientes afirmaciones sobre la palabra clave throw es verdadera?
R =d.Se utiliza para generar una excepción manualmente.
throw lanza explícitamente una excepción.
throw es la acción de lanzar; throws es el aviso en la firma del método.

6° In Java the difference between throws and throw Is:
R = c.Throws indicates the type of exception that the method does not handle and throw an exception.
throws declara en la firma; throw lanza desde el cuerpo. Ambos pueden usarse en métodos y constructores.

7° Which of the following statements correctly identifies checked and unchecked exceptions in Java?
R = b.IOException is a checked exception, and NullPointerException is an unchecked exception.
RuntimeException y sus subclases son unchecked. IOException y SQLException son checked: debes capturarlas o declararlas.

8° What is the result?
R = b.ACD
Si el try termina sin excepción, se omite el catch, se ejecuta el finally y después continúa el código siguiente.

9° ¿Qué ocurrirá si ejecutas el siguiente código?
R = c.Se imprime "Excepción capturada: Edad no válida para votar."
Unchecked no significa “sin capturar”: significa que no es obligatorio capturarla o declararla. Al ejecutarse throw, se abandona el flujo normal y se busca un catch compatible.

10° Which three are bad practices?
R = b.Checking for ArrayIndexOutOfBoundsException and ensuring that the program can recover if one occurs.
d.Checking for Error and, if necessary, restarting the program to ensure that users are unaware of problems.
e.Checking for ArrayIndexOutOfBoundsException when iterating through an array to determine when all elements have been visited.
Valida los índices y usa array.length para recorrer arreglos; no uses excepciones para controlar el flujo normal del programa.

11° ¿Cuáles de las siguientes excepciones deben ser chequeadas?
R = a. IOException
c. InterruptedException
IOException e InterruptedException son checked. ClassCastException y las demás subclases de RuntimeException son unchecked.

12° Which of the following are unchecked exceptions?
R = a.ArrayIndexOutOfBoundsException
c.RuntimeException
d.NumberFormatException
e.IllegalArgumentException
Todas las subclases de RuntimeException, directas o indirectas, son unchecked. NumberFormatException e IllegalArgumentException están incluidas.

13° ¿Cuál será la salida del siguiente fragmento de código?
R = a.Error: División por cero.
c.Ejecución finalizada.
Una división entera entre cero lanza ArithmeticException. Se ejecuta solo el primer catch compatible y después el finally.

14° ¿Cuál es la salida del siguiente código?
R = b.Running Problem Stopping
Si capturas una excepción checked dentro del método, no necesitas declararla con throws. System.err imprime en otro canal; no lanza una excepción.

15° What will be the output of the following code snippet?
R = b.Resource used Resource closed.
Try-with-resources llama automáticamente a close() al salir del bloque. El recurso debe implementar AutoCloseable; primero se usa y después se cierra.

16° Which two possible outputs?
R = a. Before if clause Exception in thread "main" java.lang.Exception at Main.doSomething (Main.java:21) at Main.main (Main.java:15).
d.Before if clause After if clause.
throw interrumpe el flujo desde donde se ejecuta; no borra lo que ya se imprimió. throws declara la posible propagación, pero no captura.

17° Which two actions, used independently, will permit this class to compile?
R = b.Adding throws IOException to the doSomething() method signature
c.Adding throws IOException to the main() method signature
Cada método debe capturar o declarar las checked que pueden salir de él. Poner throws solo en main() no resuelve una excepción sin declarar dentro de otro método.

18° ¿Cuáles son tres malas prácticas?
R = a. Comprobación de ArrayIndexOutOfBoundsException al iterar a través de una matriz para determinar cuándo se han visitado todos los elementos.
b.Verificar errores y, si es necesario, reiniciar el programa para garantizar que los usuarios no se den cuenta de los problemas.
d.Comprobar si hay ArrayIndexOutOfBoundsException y garantizar que el programa pueda recuperarse si ocurre alguna.
Prevén los índices inválidos; no uses excepciones para terminar bucles. Maneja los fallos recuperables e informa del problema.

19° ¿Qué muestra en pantalla al ejecutar este código?
R = c.A java.lang.StackOverflowError is thrown.
El código compila, pero go() se llama a sí mismo una y otra vez sin una condición de parada. Cada llamada ocupa espacio en la pila hasta que esta se agota.

20° Which three implementations are valid?
R = a. class Test implements SampleCloseable { public void close() throws java.io.IOException { // do something } }
c.class Test implements SampleCloseable { public void close() throws FileNotFoundException { // do something } }
e. class Test implements SampleCloseable { public void close() { // do something }}
Al implementar o sobrescribir: misma checked, una más específica o ninguna; nunca una más general ni una checked ajena al contrato. Para implementar una interfaz se usa implements.

21° What is the result?
R = b.La compilación falla
Un catch de una checked específica exige que su try pueda lanzarla. El throws del método exterior no cumple ese requisito.

22° What is the result if you try to compile Truthy.java and then run it with assertions enabled?
R = a.Truthy.java does NOT compile.
assert condicion : mensaje; exige una condición booleana. Si es falsa y las aserciones están habilitadas, lanza AssertionError.

10 · Colecciones y genéricos.

1° In the Java collections framework a Set Is:
R = b.A collection that cannot contain duplicate elements.
List permite duplicados; Set no. En Map, las claves son únicas, pero los valores pueden repetirse.

2° ¿Cuál de las siguientes afirmaciones sobre ArrayList es verdadera?
R = a.ArrayList permite elementos duplicados.
ArrayList permite duplicados, mantiene una secuencia y usa un arreglo interno. Para números enteros utiliza Integer, no int, como argumento de tipo.

3° ¿Cuál es la característica que comparten Array y Arraylist?
R = c.Es posible que tengan valores duplicados.
Los arrays y los ArrayList permiten duplicados. Mantener posiciones **no significa ordenar automáticamente** los valores de menor a mayor.

4° ¿Cuál de las siguientes no es parte del framework de Java Collections?
R = c.Dictionary
Map pertenece al framework Java Collections, aunque no extiende Collection. Dictionary es una clase antigua reemplazada conceptualmente por Map.

5° ¿Qué método en java.lang.Object debe ser sobrescrito si se sobrescribe el método hashCode()?
R = d.equals()
equals() define igualdad; hashCode() debe respetarla: **objetos iguales ⇒ mismo hash**. compareTo() define orden.

6° What is the output?
R = a.Apple Banana Cherry Size: 3 Contains 'Banana': true Element at index 1: Banana
En una lista, size() cuenta elementos y get() utiliza índices desde 0: con tres elementos, los índices válidos son 0, 1 y 2.

7° Dado el siguiente código, ¿cuál es el resultado?
R = b.[3, 5, 6, 8]
Collections es una clase de utilidades para trabajar con colecciones. Su método estático sort(lista) modifica la lista y ordena sus elementos según su orden natural. Para Integer, ese orden es numérico ascendente.

8° ¿Cuál de las siguientes implementaciones de código utiliza correctamente una clase genérica en Java?

R = c. public class Box<T> {

```
    private T value;
    public Box(T value) {
        this.value = value;
    }
    public T getValue() {
        return value;
    }
}
```

d. public class Box<T extends Number> {

```
    private T value;
    public Box(T value) {
        this.value = value;
    }
    public T getValue() {
        return value;
    }
}
```

Tanto <T> como <T extends Number> definen clases genéricas. Usar Object sin declarar un parámetro de tipo no hace genérica a una clase.

9° ¿Qué clase puede proveer las siguientes capacidades?: no permitir valores duplicados, mantener el orden en el cual los elementos fueron insertados, proveer iteración rápida y acceso aleatorio eficiente.

R = b.LinkedHashSet

LinkedHashSet combina elementos únicos + orden de inserción + búsqueda eficiente por valor. No proporciona acceso por índice.

10° ¿Cuál será el resultado de intentar compilar y ejecutar el siguiente fragmento de código?

R = d.Imprimirá los artículos ordenados por nombre.

Collections.sort(lista) utiliza el orden natural definido por compareTo(). El campo que se compara determina el criterio de ordenación, no los demás atributos del objeto.

11° What is the output?

R = a.Apple: 3 Banana: 2 Cherry: 1

Para saber cómo se ordenan los objetos, revisa qué compara compareTo(). Ordenar cambia la posición de los objetos, no sus atributos.

12° What is the result?

R = d.7 5 3 2

Arrays.sort(arreglo, comparador) modifica el arreglo usando la regla del comparador. Para enteros, y.compareTo(x) ordena de mayor a menor.

13° What is the result?
R = d.[70, 50, 30, 20]
Comparator<Long>.compare() devuelve **int**, no long. Con b.compareTo(a), los valores se ordenan de mayor a menor.

14° Dado el siguiente código. ¿Cuál es el resultado?
R = c.8,6,4,1
Arrays.sort(arreglo, Collections.reverseOrder()) modifica el arreglo ordenándolo de mayor a menor; no conserva el orden original ni simplemente lo invierte.

15° Dado el siguiente código. ¿Cuál es el resultado?
R = a.Nadia, Mariel, Elo, Alejandro
Collections.sort(lista, Collections.reverseOrder()) ordena la lista en orden natural inverso.

16° ¿Cuáles de los siguientes fragmentos de código implementan correctamente Comparator para ordenar una lista de objetos Person por su edad en orden ascendente? (Elige dos)
R = a.import java.util.Comparator;

```
public class AgeComparator implements Comparator<Person> {
    @Override
    public int compare(Person p1, Person p2) {
        return p1.getAge() - p2.getAge();
    }
}
```

```
c.import java.util.Comparator;
public class AgeComparator implements Comparator<Person> {
    @Override
    public int compare(Person p1, Person p2) {
        return Integer.compare(p1.getAge(), p2.getAge());
    }
}
```

Para ordenar por edad ascendente, usa Integer.compare(p1.getAge(), p2.getAge()). Primero contra segundo: ascendente; segundo contra primero: descendente.

17° What is the result?
R = c.[7, 5, 1]
En una List, remove(1) elimina por índice; remove(Integer.valueOf(1)) elimina la primera aparición del valor 1.

18° What is the result?
R = c.The output is true and MyStuff does NOT fulfill the Object.equals() contract.
Cuando equals() está sobrescrito, sigue el cuerpo real del método. Que compile y devuelva un resultado no garantiza que cumpla el contrato de igualdad.

19° What is the result?

R = c.4

Si las claves de un HashMap tienen igualdad personalizada, implementa equals() y hashCode() coherentemente. Para comparar el valor de dos Integer, utiliza equals() u Objects.equals(), no ==.

11 · Lambdas y Streams.

Interfaz	Recibe	Devuelve	Método	Ejemplo
Predicate<T>	Un T	boolean	test(T)	x -> x > 5
Function<T, R>	Un T	Un R	apply(T)	s -> s.length()
Consumer<T>	Un T	Nada: void	accept(T)	s -> System.out.println(s)
Supplier<T>	Nada	Un T	get()	() -> "Hola"
UnaryOperator<T>	Un T	Un T	apply(T)	x -> x * 2
BinaryOperator<T>	Dos T	Un T	apply(T, T)	(a, b) -> a + b

1° What is the output?
R = c.[apple]
filter(Predicate) conserva los elementos cuya condición devuelve true; startsWith("a") comprueba el inicio, no si la letra aparece en cualquier posición.

2° Dado el siguiente código. ¿Cuál es el resultado?
R = b.Andrea, Josias, Kevin, Laila
sorted() ordena por orden natural sin modificar la colección original. Antes de calcular la salida, comprueba si el código compila y si el ejercicio presupone los imports.

3° Dado el siguiente código. ¿Cuál es el resultado?
R = a.9, 8, 3, 1
sorted(Collections.reverseOrder()) ordena en sentido natural inverso; no se limita a invertir las posiciones originales ni modifica la lista fuente.

4° ¿Cuál es el resultado del siguiente código?
R = b. apple banana grapes pears
(a, b) -> a.compareTo(b) define el orden natural ascendente. El comparador devuelve un int, y Arrays.sort() modifica directamente el arreglo original.

5° ¿Cuál es el resultado al ejecutar el siguiente código?
R = c.3
Las operaciones se aplican en el orden escrito: map transforma, filter evalúa el valor transformado y count() cuenta los elementos que sobreviven.

6° Dado el siguiente código: ¿Qué imprime en pantalla el código?
R = e.[Mark]
filter conserva los elementos cuyo predicado devuelve true. Con !condición, primero evalúa la condición y luego invierte el resultado: true → false y false → true.

7° What will be the output of the above code?
R = e.true 10 FUNCTIONAL Hello, world!
Predicate.test() comprueba; Function.apply() transforma; Consumer.accept() actúa sin devolver; Supplier.get() proporciona sin recibir argumentos.

8° ¿Cuál será la salida del código anterior?
R = d.false
p.or(q).and(r) se agrupa como (p OR q) AND r. Devuelve un nuevo predicado; reasignarlo a p es válido si las lambdas no capturan esa variable local.

9° What will be the output of the above code?
R = a.2
distinct() elimina duplicados según equals(), no solo según ==. Dos instancias distintas pueden contar como un único elemento si son iguales por valor y respetan el contrato de hashCode().

10° Given: What will be the output of the above code?
R = e.Compilation fails on line 3.
Los parámetros de una lambda tienen alcance propio, pero no pueden reutilizar el nombre de una variable local que ya esté en alcance en el método contenedor.

12 · Fechas, hilos y JVM.

1° ¿Cuál es la salida del siguiente código?
R = c.Se genera una excepción en tiempo de ejecución.
LocalDate.of(año, mes, día) exige una fecha válida: si no existe, lanza DateTimeException en ejecución; no ajusta los valores automáticamente.

2° What is the result?
R = c.A java.lang.StackOverflowError is thrown.
La recursión sin caso base acumula llamadas y puede provocar StackOverflowError. En array[x % 5] = x++, primero se calcula el índice y se almacena el valor anterior al incremento.

3° What is the result?
R = d. An exception is thrown at runtime.
nextInt() exige que el siguiente token sea un entero válido; no busca números más adelante. hasNextInt() comprueba sin consumir y next() permite descartar un token.

4° What is the result?
R = b.The program prints: 1 thrown to main.
Para llamar a obj.wait(), el hilo actual debe poseer el monitor de ese mismo obj; de lo contrario, se lanza IllegalMonitorStateException. Un método de instancia synchronized bloquea this.

5° Which of the following will be the output? (Choose two.)
R = d.The code outputs 1 3 4 2.
f.The code runs to completion.
wait() libera el monitor; sleep() lo conserva; notify() avisa, pero el hilo despertado debe recuperar el monitor antes de continuar. Usa wait() dentro de un while que compruebe la condición.

6° What is the result?
R = b.111.234 222.5678
setMaximumFractionDigits() afecta a format(), no a parse(). throws declara una posible excepción; no obliga a que ocurra.

7° ¿Cuál es la respuesta verdadera?
R = c.La salida nunca puede contener el valor 1130.
Con hilos, distingue el máximo de operaciones posibles del resultado real. i++ no es atómico y yield() no garantiza orden ni sincronización.

8° Assume that the default locale formats FormatStyle.SHORT date-times as dd/MM/yy HH:mm. What is the result?
R = g.12/03/96 23:55
Period.of(años, meses, días) opera sobre la fecha. LocalDateTime es inmutable: guarda el resultado de minus() o plus(). El formato cambia lo que se muestra, no el valor almacenado.